smaller scale, map-reduce is just a clean, manageable way to structure data management. The Ruby Enumerable module provides support for both parts of the pattern.

The map method (which is aliased to collect for historical reasons) takes a block and applies the block to each element in turn, returning a new array with the block's return value for each element:

```
x = [1, 2, 3, 4]
x.map { |element| element * 11 }  # => [11, 22, 33, 44]
```

The reduce method (which is aliased to inject, again for historical reasons) is a way to convert a collection of values into a single value. There are a couple of different ways reduce works. The most general takes both optional initial value as an argument and a block.

The block takes two arguments, the first of which is the accumulator, and the second of which is each element in the list in turn. If the reduce method passed the initial value as an argument, then that value is used as the value of the accumulator on the first call to the block. If not, the first call to the block uses the first element of the list as the accumulator, and the second element of the list as the element argument.

Inside the block, the idea is to do something to update the accumulator with the new value and return the new accumulated value, which is then passed forward to the next iteration of the block:

```
x = [2, 5, 9, 12]
x.reduce { |sum, element| sum + element }                              # => 28

words = %w(major league baseball)
words.reduce("") { |acronym, element| acronym << element[0].upcase }  # => "MLB"
```

In the first example, there's no starter value, so the block is called with arguments 2 and 5 from the array, resulting in 7. The block is then called with that 7 as the first argument and the the third element of the list—9—as the next argument. The result is 16 and the next call to the block is 16 and 12, so the final result is 28.

In the second example, there is a starter value so the first block is called with "" and "major". The result of that block is "M", so the next call is "M" and league, and so on until the final result is "MLB".

The reduce method also has a form that is somewhat unusual in the Ruby libraries. It takes a second argument that is a symbol, where the symbol is a method that takes two arguments. It then uses that method as if it was the body of the block. In other words, the following lines are equivalent:

```
x = [2, 5, 9, 12]
x.reduce { |sum, element| sum + element } # => 28
x.reduce(0, :+)                           # => 28
x.sum                                     # => 28
```

Here, + is a method that takes two arguments, and so the reduce method treats it as body of a block.

In fact, the sum pattern is so common that Ruby eventually added a sum method, as you can see in the final line. The sum method takes its own optional block, in which case it sums the return values of the block. In other words, the following lines are equivalent: